18 Post Mortems

Hindsight is 20-20

Unknown

This chapter is meant to serve as a big "why are we learning all of this". In all of your previous classes, you were learning what to do. How to program a data structure, how to code a for loop, how to prove something. This is the first class that is largely focused on what *not* to do. As a result, we draw experience from our past in real ways. Sit back and scroll through this chapter as we tell you about the problems of past programmers. Even if you are dealing with something much higher level like web-development, everything relates back to the system.

18.1 Shell Shock

Required: Appendix/Shell

This was a back door into most shells. The bug allowed an attacker to exploit an environment variable to execute arbitrary code.

```
$ env x='() { :;}; echo vulnerable' bash -c "echo this is a test"
vulnerable...
```

This meant that in any system that uses environment variables and doesn't sanitize their input (hint no one sanitized environment variable input because they saw it as safe) you can execute whatever code you want on other's machines including setting up a web server.

Lessons Learned: On production machines make sure that there is a minimal operating system (something like BusyBox with DietLibc) so that you can understand most of the code in the systems and their effectiveness. Put in multiple layers of abstraction and checks to make sure that data isn't leaked. For example the above is a problem insofar as getting information back to the attackers if it is allowed to communicate with them. This means that you can harden your machine ports by disallowing connections on all but a few ports. Also, you can harden your system to never perform exec calls to perform tasks (i.e. perform an exec call to update a value) and instead do it in C or your favorite programming language of choice. Although you don't have flexibility, you have peace of mind about what you allow users to do.

18.2 Heartbleed

Required: Intro to C

To put it simply, there were no bounds on buffer checking. The SSL Heartbeat is super simple. A server sends a string of a certain length, and the second server is supposed to send the string of that length back. The problem is someone can maliciously change the size of the request to larger than what they sent (i.e. send "cat" but request 500 bytes) and get crucial information like passwords from the server. There is a Relevant XKCD on it.

Lessons Learned: Check your buffers! Know the difference between a buffer and a string.

18.3 Dirty Cow

Required: Processes/Virtual Memory

Dirty Cow

A process usually has access to a set of read-only mappings of memory that if they try to write to they get a segfault. Dirty COW is a vulnerability where a bunch of threads attempt to access the same piece of memory at the same time hoping that one of the threads flips the NX bit and the writable bit. After that, an attacker can modify the page. This can be done to the effective user id bit and the process can pretend it was running as root and spawn a root shell, allowing access to the system from a normal shell.

Lessons Learned: Spinlocks in the kernel are hard.

18.4 Meltdown

Meltdown is a close cousin of Spectre: both abuse out-of-order execution to leak data the program shouldn't be able to read. See Section 14.2.3 in the security chapter.

18.5 Spectre

See Section 14.2.3 in the security chapter.

18.6 Mars Pathfinder

Required sections: Synchronization and a bit of Scheduling

Pathfinder Link

The Mars Pathfinder was a 1997 mission that, among other things, collected weather data on Mars. The lander used a single "information bus" to pass data between its different parts, and access to the bus was protected by a mutex. The architecture was pretty simple. There was a bus management thread with high

priority, a communications thread with medium priority, and a meteorological data collection thread with low priority. The scheduler was preemptive: whenever a higher-priority task became ready to run, it took the CPU from any lower-priority task.

The pattern that caused everything to start failing went like this. The low-priority data collection thread locked the mutex to publish its data on the bus. The high-priority bus thread then tried to lock the same mutex and had to wait. Then the medium-priority communications thread became ready and preempted the low-priority thread **while the low-priority thread still held the mutex**. The communications thread ran for a long time, so the low-priority thread couldn't finish and release the mutex, and the high-priority bus thread stayed blocked behind both of them. This is called `priority inversion`: a medium-priority task effectively kept a high-priority task from running. After a while, a watchdog timer noticed that the bus thread hadn't run and reset the whole system, losing data each time. The fix, uploaded to the spacecraft, was to turn on `priority inheritance` for that mutex, so a low-priority thread holding the mutex temporarily runs at the priority of the highest-priority thread waiting for it.

Lessons Learned: Understand priority inversion before you mix locks and priorities. If a lock is shared by tasks with different priorities, turn on priority inheritance (or use a priority ceiling) for it. Keep your debugging and tracing hooks in the deployed system, because that's how the engineers diagnosed and patched this one from Earth.

18.7 Mars Again

Required Sections: Malloc

Mars

The short of it is that they ran out of memory. The long of it is that they ran out of memory, disk space, and swap space. The moral of the story? Make sure to write code that can handle file failures and can handle files when they close and go out of memory, so the operating system can hot swap files to free up memory. Also clean up files, assume that your temp directory is roughly a hundredth or a thousandth of the total size and use that.

18.8 Year 2038

Required sections: Intro to C

2038

This is an issue that hasn't happened yet. Unix timestamps are kept as the number of seconds from a particular day (Jan 1st 1970). This is stored as a 32 bit signed integer. On January 19, 2038 (at 03:14:07 UTC), this number will overflow. This isn't a problem for most modern operating systems that store 64 bit signed integers which is enough to keep us going until the end of time, but it is a problem for embedded devices that we can't change the internal hardware to. Stay tuned to see what happens.

Lessons learned: Plan like your application will be huge one day.

18.9 Northeast Blackout of 2003

Required Sections: Synchronization

2003

A race condition triggered a series of undefined events in a system that caused the blackout of most of the northeastern part of North America for quite some time. This bug also turned off or caused the backup system and logging systems to fail so people didn't even know of the bug for an hour. The exact bits that were flipped are unknown, but patches have been put into place.

Lessons Learned: Modularize your code to localize failures (i.e. keeping race conditions different between processes). If you need to synchronize among processes make sure your failure detection system is not interlaced with your system.

18.10 Apple IOS Unicode Handling

Required Sections: Intro to C

Crashing your iphone with text

Wonder why we teach string parsing? Because it is a hard thing to do even for professional software developers. This bug allowed a lot of undefined behavior when trying to parse a series of unicode characters. Apple probably knows why this happened, but our guess is that the parsing of the string happens somewhere inside the kernel and a segfault is reached. When you get a segfault in the kernel, your kernel panics, and the entire device reboots. Undefined behavior means anything though, and a lot of varied things did happen with this bug.

Lessons Learned: Fuzz your kernel.

18.11 Apple SSL Verification

Required Sections: Intro to C

Apple Bug

Due to a stray goto in Apple's code, a function always returned that an SSL certificate was valid. Naturally, hackers were able to get away with some pretty crazy site names.

Lessons Learned: Always bracket if statements, use gotos sparingly. Chances are if you need to use a goto, write another function or a switch statement with fall throughs (still bad).

18.12 Sony Rootkit Installation

Required Sections: Intro to C/Processes

Root Kit Scandal

Picture this. It's 2005, Limewire came a few years prior, the internet was a growing pool of illegal activities – not to say that is fixed now. Sony knew that it didn't have the computing power to police all the interwebs

or get around the various technologies people were using to get around copyright protection. So what did they do? With 22 million Music CDs, they required users to install a rootkit on their operating system, so Sony could monitor the device for unethical activities.

Privacy concerns aside, and believe me there are a lot of them, the big problem was that this rootkit was a backdoor for everyone's systems if programmed incorrectly. A rootkit is a piece of code usually installed kernel-side that keeps track of almost anything that a user does. It can see which websites are visited, which buttons are clicked, which keys are typed, etc. If a hacker finds out about this and there is a way to access that API from the user space level, that means any program can find out important information about your device. Needless to say, people were angry.

Lessons Learned: Get an antivirus and/or apparmor and make sure that an application is only requesting permissions that make sense. If you are torn, try something like Windows sandbox or keep a Sacrificial VM around to see if installing it makes your computer horrible. Don't trust certificates, trust code.

18.13 The Woes of Shell Scripting

Required Sections: Appendix/Shell

Steam

There was a simple bug in Steam that caused Steam to remove all of your files in the form of something like this

```
STEAMROOT="$(cd "${0%/*}" && echo $PWD)"
# ...
rm -rf "$STEAMROOT/"*
```

In a shell script, $0 is the path of the script itself (not the first parameter, which is $1), so the first line tries to find the directory the script lives in. What happens if that directory doesn't exist, for example because the user moved it? The `cd` fails, `STEAMROOT` is empty, and the second line deletes everything under the root directory that the user can write to.

Lessons Learned: Do parameter checks, always always always `set -e` on a script and if you expect a command to fail, explicitly list it. You can also alias rm to mv and then delete the trash later.

18.14 Appnexus Double Free

Required Sections: Intro to C/Malloc

Double Free (archived copy)

Appnexus uses an asynchronous garbage collector that reclaims different parts of the heap when it believes that objects are unused. To keep latency low, when an in-memory object is deleted, it is unlinked from other objects and its memory is scheduled to be freed at a safe time in the future, when no thread could still be using it. In 2013, a rarely-changed object was deleted by a data update, and a bug in the code deleted that object twice. The update passed validation because the actual free hadn't happened yet, so it was distributed to all of the roughly 900 ad-serving servers. When the scheduled frees finally ran, the double free crashed all of them at nearly the same time.

Lessons Learned: Avoid making hacky software if you need to. Modularize, and set memory limits, and monitor different parts of your code and optimize by hand. There is no general catch-all garbage collector that fits everyone. Even highly tested ones like the JVM need some nudges if you want to get performance out of them.

18.15 ATT Cascading Failures - 1990

Required Sections: Intro to C

Explanation (archived copy)

The bug is explained well at the link above. We recommend reading to learn more. A switch in New York reset itself after a fault and told its neighbors that it was out of service. When it came back online, it began routing the backlog of calls that had built up and sent a burst of closely timed messages to other switches. Because of a misplaced `break` statement in the C code, a switch that received a second message while still processing the first overwrote its own data and reset itself. When each of those switches came back online, it sent its own burst of messages, so the resets cascaded across all 114 switches in the network for about nine hours.

Lessons Learned: A single misplaced `break` took down a national phone network. Know exactly what `break` exits in C: the innermost `switch` or loop, never an `if`. Test your failure and recovery paths, not just the happy path, for example with simulations or fuzzing that deliver messages with random timing. Recovery code runs rarely, so it is the least tested code you have, and when it misbehaves on many machines at once it can turn one failure into a cascade.